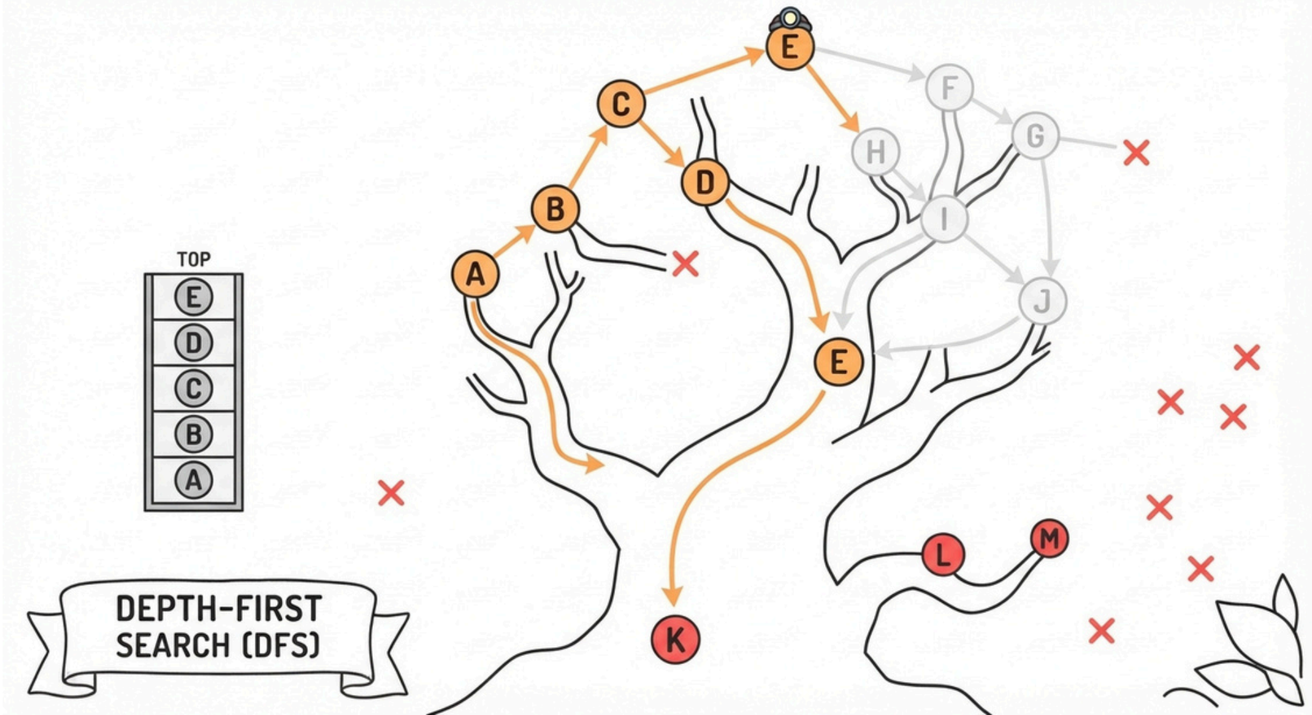
DFS, BFS, UCS ЖӘНЕ A* ІЗДЕУ АЛГОРИТМДЕРІН САЛЫСТЫРУ

DFS(DEPTH-FIRST SEARCH)

бұл графты айналып шығу алгоритмі, ол мүмкін болғанша рекурсивті түрде «тереңге» қарай барады, содан кейін ғана кері оралады. Ол ең қысқа жолдарды іздеу, топологиялық сұрыптау және циклдерді табу сияқты тапсырмаларды шешуге қолданылады.



DFS(DEPTH-FIRST SEARCH)

```
friends = {
    'Алия': ['Данияр', 'Ерлан'],
    'Данияр': ['Алия', 'Сабина'],
    'Ерлан': ['Алия', 'Асель'],
    'Сабина': ['Данияр'],
    'Асель': ['Ерлан', 'Марат'],
    'Марат': ['Асель']
}
def dfs(graph, person, target, path=None):
    if path is None:
        path = [person]

    print(person)

    if person == target:
        return path

    for friend in graph[person]:
        if friend not in path:
            result = dfs(graph, friend, target, path + [friend])
            if result:
                return result

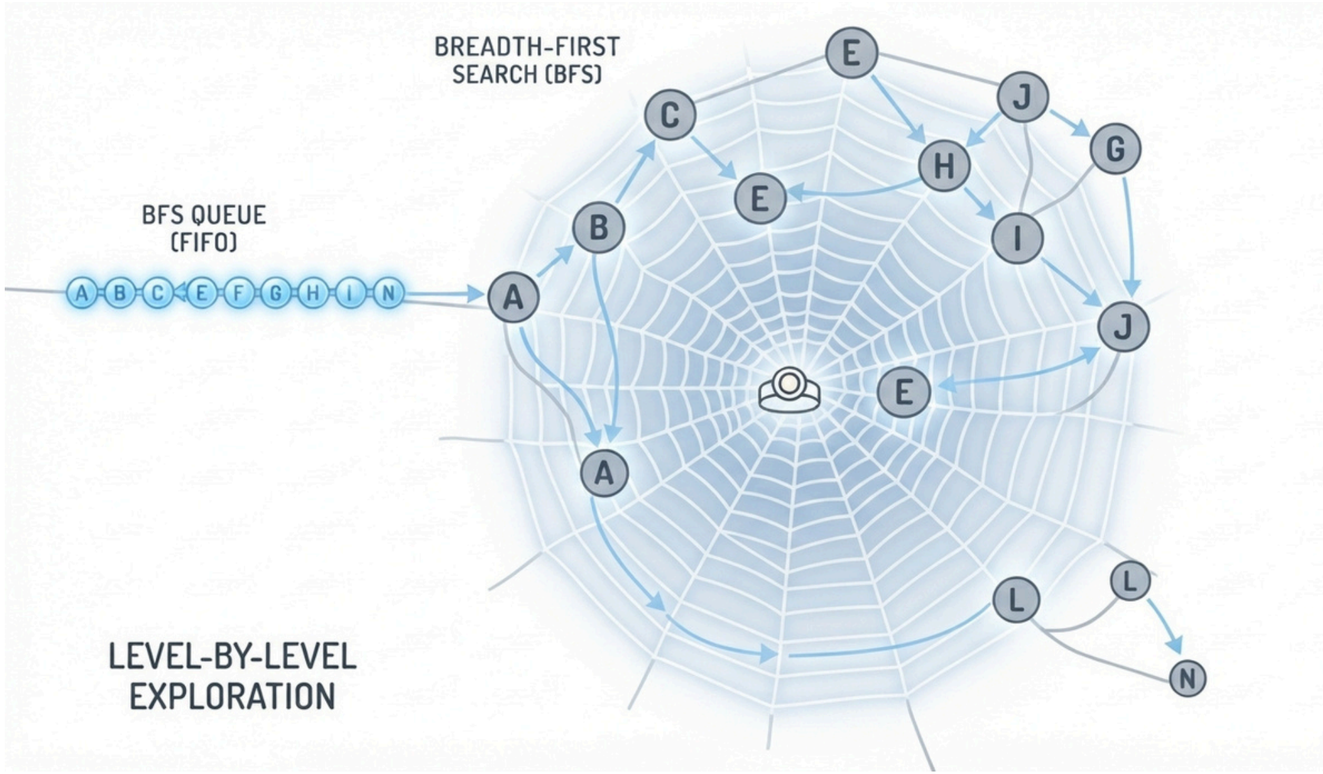
    return None

route = dfs(friends, 'Сабина', 'Марат')
print("Таныстық тізбегі:", route)
```

Мысалға біздегі код ол Достар тізбегі,яғни бізге Сабинадан Маратқа баруымыз қажет.Код DFS болғандықтан бірінші Сабина тармақтарын тереңнен қарайды қандай тармақ ең жақын соны таңдап,сол тармақтан тереңнен іздейді.
сонымен код осылай жұмыс жасайды:

Сабина > Данияр > Алия > Ерлан > Асель > Марат

BFS(BREADTH-FIRST SEARCH)



Бұл графты өту алгоритмі, ол бастапқы төбеден бастап графтың барлық төбелерін жүйелі түрде аралап шығып, әрқайсысына дейінгі қашықтықты есептейді.

Негізгі принциптер:
Алгоритм қабат бойынша жұмыс істейді, әр төбенің көршілерін аралап шығады, мақсатты төбеге жеткенге дейін.

BFS(BREADTH-FIRST SEARCH)

```python
friends = {
    'Алия': ['Данияр', 'Ерлан'],
    'Данияр': ['Алия', 'Сабина'],
    'Ерлан': ['Алия', 'Асель'],
    'Сабина': ['Данияр'],
    'Асель': ['Ерлан', 'Марат'],
    'Марат': ['Асель']
}
from collections import deque

def bfs(graph, start, target):
    visited = set([start])
    queue = deque([[start]])

    while queue:
        path = queue.popleft()
        person = path[-1]

        print(person)

        if person == target:
            return path

        for friend in graph[person]:
            if friend not in visited:
                visited.add(friend)
                queue.append(path + [friend])

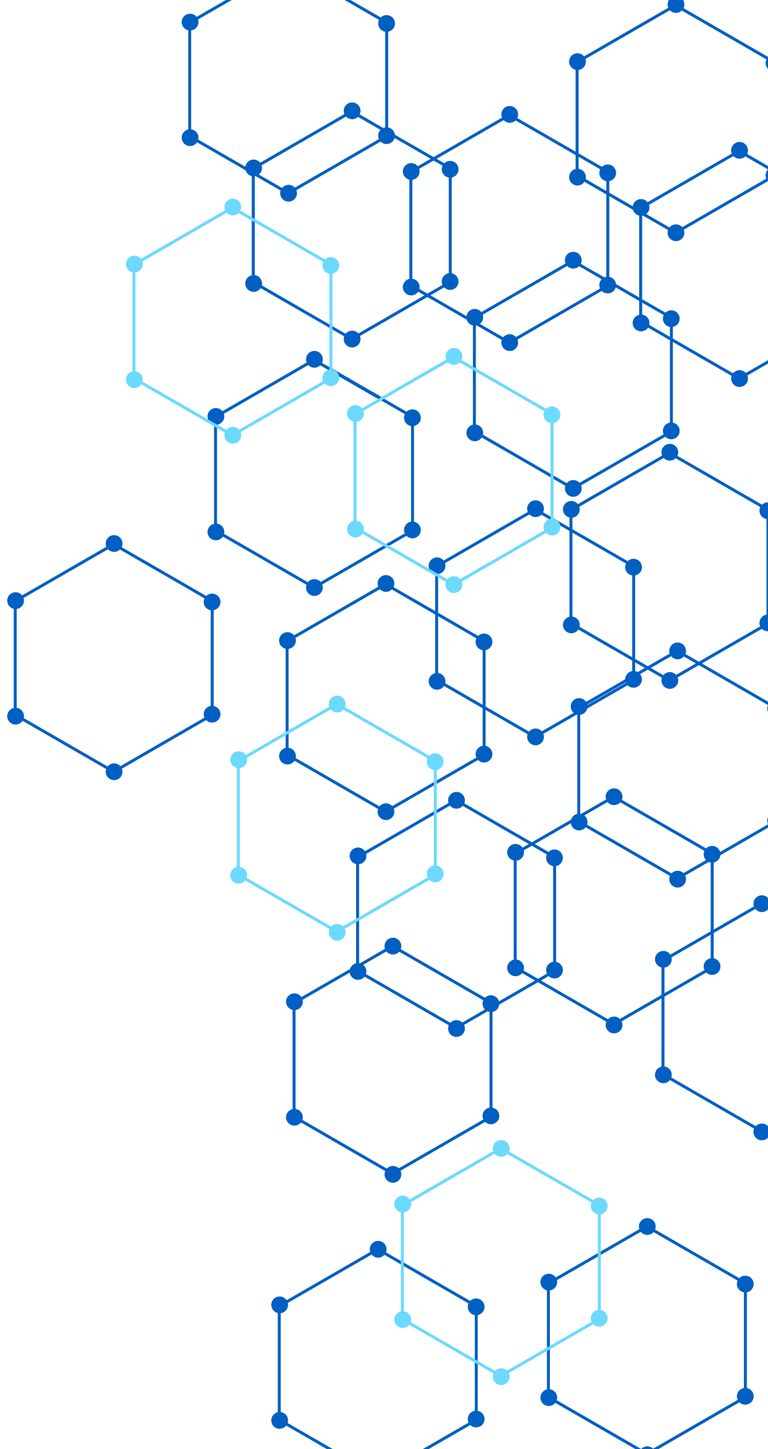
    return None

route = bfs(friends, 'Сабина', 'Марат')
print("Тізбек:", route)
```

Бұл жерде мысалға достар желісі BFS кодымен жазылған.Код бізге Сабинадан Маратқа дейін іздейді. Бұл кодта DFS-тен айырмашылығы,DFS алгоритмы тереңінен яғни тармақтардың барлығын көріп ең жақынын алады.
Ал BFS болса деңгейімен жүреді.Алдымен бірінші деңгейді қарап шығады сосын қайсысы жақынырак соны тандап келесі деңгейді қарап шығады осылай жұмыс жасайды.

UCS(Uniform Cost Search)

бұл ақпаратсыз іздеу алгоритмі, ол салмақталған графтағы бастапқы түйін мен мақсатты түйін арасындағы ең аз шығын жолын табады. Іздеудің тереңдігін басымдыққа беретін әдістерге қарағанда, UCS әрбір түйінге жету құнын есептейді, бұл әртүрлі әрекеттердің әртүрлі шығындары болатын тапсырмалар үшін пайдалы етеді.



UCS(UNIFORM COST SEARCH)

```python
friends = {
    'Алия': [('Данияр', 1), ('Ерлан', 4)],
    'Данияр': [('Алия', 1), ('Сабина', 2)],
    'Ерлан': [('Алия', 4), ('Асель', 1)],
    'Сабина': [('Данияр', 2)],
    'Асель': [('Ерлан', 1), ('Марат', 3)],
    'Марат': [('Асель', 3)]
}
import heapq

def ucs(graph, start, target):
    # приоритет кезегі: (жиынтық_құн, жол)
    queue = [(0, [start])]
    visited = set()

    while queue:
        cost, path = heapq.heappop(queue)  # ең арзан жолды аламыз
        person = path[-1]

        if person == target:
            return cost, path

        if person in visited:
            continue
        visited.add(person)

        print(f"{person} (құны: {cost})")

        for friend, weight in graph[person]:
            if friend not in visited:
                heapq.heappush(queue, (cost + weight, path + [friend]))

    return None, None

total_cost, route = ucs(friends, 'Сабина', 'Марат')
print("Ең арзан тізбек:", route)
print("Жалпы құны:", total_cost)
```

UCS алгоритмнің басқалардан ең басты айырмашылығы ол тармақтардың арасындағы салмақтарды есептейді.
Нақтылай келсек екі тармақ арасындағы ең аз салмақты таңдайды.

Бұл кодта Сабинадан Маратқа бару үшін оған дейінгілерден қарағанда араларындағы ең арзан салмақ соны таңдап жүріп отырады

A* ALGORITM

Жұмыс принципі: A* приоритет функциясын $f(n) = g(n) + h(n)$ пайдаланады, мұнда:

$g(n)$ — бастапқы нүктеден n түйініне дейінгі жолдың құны (Дейкстрадағыдай);

$h(n)$ — n-ден мақсатқа дейінгі арақашықтықтың эвристикалық бағасы (мысалы, Манхэттен немесе Евклид қашықтығы)

бұл графтарда ең қысқа жолды табудың ең танымал әдістерінің бірі, әсіресе навигация және ойындарды дамыту тапсырмаларында. Ол Дейкстра алгоритмінің артықшылықтарын (жолдың нақты құнын ескеру) және ашкөз іздеудің артықшылықтарын (мақсатқа жететін бағыттарға басымдық беру) біріктіреді, бұл оны дұрыс эвристикамен жылдам және тиімді етеді.

A* ALGORITM

UCS бізде тек ең арзан салмақтарды таңдап жүрсе,
A* барлық салмақтарды есептеп алып ең тиімдісімен жүреді.
Және де A* болжауды колдана жүреді,тез — мақсатқа "бағыттап" іздейді, артық тармақтарды аз қарайды



```
friends = {
    'Алия': [('Данияр', 1), ('Ерлан', 4)],
    'Данияр': [('Алия', 1), ('Сабина', 2)],
    'Ерлан': [('Алия', 4), ('Асель', 1)],
    'Сабина': [('Данияр', 2)],
    'Асель': [('Ерлан', 1), ('Марат', 3)],
    'Марат': [('Асель', 3)]
}
heuristic = {
    'Алия': 5,
    'Данияр': 7,
    'Ерлан': 3,
    'Сабина': 9,
    'Асель': 3,
    'Марат': 0   # мақсаттың өзіне дейінгі қашықтық әрқашан 0
}
import heapq

def a_star(graph, heuristic, start, target):
    # приоритет кезегі: (f, g, жол)
    queue = [(heuristic[start], 0, [start])]
    visited = set()

    while queue:
        f, g, path = heapq.heappop(queue)
        person = path[-1]

        if person == target:
            return g, path

        if person in visited:
            continue
        visited.add(person)

        print(f"{person} (g={g}, h={heuristic[person]}, f={f})")

        for friend, weight in graph[person]:
            if friend not in visited:
                new_g = g + weight
                new_f = new_g + heuristic[friend]
                heapq.heappush(queue, (new_f, new_g, path + [friend]))

    return None, None

total_cost, route = a_star(friends, heuristic, 'Сабина', 'Марат')
print("Ең тиімді тізбек:", route)
print("Жалпы құны:", total_cost)
```